Inference cost of TransNet V2 and PERSIST, measured with HybridSBD and AutoShot

Measurement only: nothing was trained and no dataset was used. The same protocol was run on three servers with the same GPU (RTX PRO 6000 Blackwell Workstation Edition) and different hosts. **Server 3 reproduces the earlier session's values and is the one to use for Table 4**; servers 1 and 2 show which results depend on the host.

1. Result for Table 4 (server 3)

FP32, `model.eval()`, `torch.no_grad()`, no AMP, no `torch.compile`; synthetic clips in each model's native format; 10 warm-up and 80 timed forward passes with `torch.cuda.synchronize()` around each; latency is mean ± SD (ms); GFLOPs per clip at B = 1 (`torch.utils.flop_counter.FlopCounterMode`: convolutions and matrix multiplications only). The two right-hand columns give the model's latency divided by the HybridSBD latency at B = 16.

Model	Input (T × H × W, dtype)	Parameters	GFLOPs / clip	B = 1	B = 16	B = 32	frames/s (B = 16)	Peak VRAM (B = 16, MB)	× A0	× A4
TransNet V2	128 × 27 × 48, uint8 RGB	7,616,258	106.50	2.6 ± 0.01	50.3 ± 0.03	99.3 ± 0.07	40,712	2,692	1.7	4.9
PERSIST (T = 128)	128 × 112 × 112, uint8 RGB (resized to 27 × 48 inside the model)	13,195,594	158.94	5.5 ± 0.01	74.4 ± 0.72	150.8 ± 1.50	27,536	2,913	2.6	7.3
AutoShot	128 × 27 × 48, float32 RGB	14,299,202	96.83	9.7 ± 0.06	160.1 ± 1.32	319.8 ± 1.18	12,792	16,205	5.6	15.6
HybridSBD A0 (D = 384, L = 4)	128 × 48 × 64, float32 RGB	10,938,880	29.43	4.0 ± 0.11	28.8 ± 0.02	62.2 ± 0.02	71,120	1,749	1.0	2.8
HybridSBD A4 (reduced)	128 × 48 × 64, float32 RGB	8,983,570	10.79	2.1 ± 0.02	10.2 ± 0.01	21.2 ± 0.01	200,182	977	0.4	1.0
PERSIST (native T = 64)	64 × 112 × 112, uint8 RGB (resized to 27 × 48 inside the model)	13,195,594	79.47	4.1 ± 0.01	36.7 ± 2.01	75.6 ± 0.86	27,913	1,499	1.3	3.6

- HybridSBD A0 is 1.7× faster than TransNet V2, 2.6× faster than PERSIST and 5.6× faster than AutoShot at B = 16, with 3.3–5.4× fewer operations than those three (29.4 against 96.8–158.9 GFLOPs); A4 is 4.9×, 7.3× and 15.6× faster.
- PERSIST accepts a 128-frame clip, so the main row uses T = 128 like the other models; its native window of 64 frames is the last row (frames per second stay comparable).
- Only HybridSBD, TransNet V2 and PERSIST latencies are independent of the host (section 3); AutoShot's latency and every B = 1 latency depend on it.

2. Setup

	Server 1	Server 2	Server 3
GPU	RTX PRO 6000 Blackwell WS, idle	same	same
CPU	AMD EPYC 9555	AMD Threadripper PRO 7975WX	Intel Core i9-14900K
CPU frequency	cpufreq maximum 1.9 GHz (`schedutil`)	rated 5.35 GHz, **capped at 2.3 GHz** by the host (`scaling_max_freq`, read-only), cores at 2.28 GHz	not capped: P-cores sampled at a median of 5.97 GHz, maximum 6.00 GHz
Benchmark process	free scheduling	free scheduling	pinned to the performance cores (CPUs 0–15)
PyTorch / CUDA / cuDNN	2.14.1+cu132 / 13.2 / 9.24	2.11.0+cu128 / 12.8 / 9.19	2.14.1+cu132 / 13.2 / 9.24
Python	3.12.3 (system)	3.12.15 (Vast image)	3.12.15 (Vast image)
Driver	590.48.01	570.195.03	610.57.04

Identical on all three: TransNet V2 `soCzech/TransNetV2` @ `85cef72af9a9` (PyTorch implementation in `inference-pytorch/`), PERSIST `linty5/PERSIST` @ `1b38eb3bc057` (MMAction2 v1.2.0 fork; mmengine 0.10.7, mmcv-lite 2.1.0, einops 0.8.2), AutoShot `wentaozhu/AutoShot` @ `77c82ff826a9`; the benchmark code; the A0 and A4 `best_model.pt` (SHA-256 prefixes `8ec3feb706b601ed`, `0125c80ffa1a6b79`, checked on each server); random weights for the three baselines. Server 2's driver (570) cannot run CUDA 13, so it used its preinstalled PyTorch 2.11.0+cu128; server 1 went offline before it could be repeated with that version, so servers 1 and 2 differ in CPU and in software.

3. All three servers

Latency in ms (mean). S1, S2, S3 are the servers above.

Model	B = 1: S1	S2	S3	B = 16: S1	S2	S3	B = 32: S1	S2	S3	Spread of B = 16 (S1–S3)
TransNet V2	3.0	4.0	2.6	51.4	55.4	50.3	101.9	113.3	99.3	10 %
PERSIST (T = 128)	14.6	7.9	5.5	75.3	77.3	74.4	152.1	158.7	150.8	4 %
AutoShot	15.5	18.9	9.7	241.8	288.8	160.1	481.3	582.0	319.8	80 %
HybridSBD A0 (D = 384, L = 4)	7.8	12.3	4.0	29.3	29.1	28.8	62.5	62.0	62.2	2 %
HybridSBD A4 (reduced)	4.1	6.6	2.1	10.6	11.0	10.2	21.4	22.2	21.2	7 %
PERSIST (native T = 64)	16.5	8.1	4.1	38.4	38.0	36.7	76.6	78.7	75.6	5 %

GPU kernel time and GPU-busy fraction of one forward pass at B = 16 (profiler); peak VRAM at B = 16 (MB).

Model	Wall-clock S1	S2	S3	GPU kernel S1	S2	S3	GPU busy S1	S2	S3	Peak VRAM S1 / S2 / S3
TransNet V2	50.8	56.2	50.5	50.5	55.4	50.6	99 %	99 %	100 %	2,692 / 2,671 / 2,692
PERSIST (T = 128)	75.3	77.6	74.8	72.5	74.3	73.9	96 %	96 %	99 %	2,913 / 2,890 / 2,913
AutoShot	276.1	291.8	159.5	86.5	87.5	86.7	31 %	30 %	54 %	16,205 / 16,183 / 16,205
HybridSBD A0 (D = 384, L = 4)	29.4	29.5	28.9	28.6	28.4	28.9	97 %	96 %	100 %	1,749 / 1,702 / 1,749
HybridSBD A4 (reduced)	10.6	11.0	10.3	10.1	10.5	10.2	95 %	96 %	99 %	977 / 953 / 977

- **A0, A4, TransNet V2 and PERSIST are GPU-bound** (95–100 % GPU busy): B = 16 and 32 latencies differ by at most 10 % between the servers, the GPU kernel time is the same on all, and the ranking and speedup factors are the same.
- **B = 1 follows the host**, in both directions (A0: 7.8, 12.3 and 4.0 ms; PERSIST: 14.6 with SD 17.5, 7.9 and 5.5 ms). At B = 1 the kernels are tiny and launch and synchronisation overhead dominate, so single-batch latencies should be reported with the machine named.
- **AutoShot follows the host at every batch size** (B = 16: 242, 289 and 160 ms) while its GPU kernel time is about 87 ms on all three; it is GPU-busy only 30–54 % of the time.

4. Reproduction of the expected values

Expected values are the paper's earlier measurements (B = 16). Each cell shows the measured value and the difference from the expected one.

Model	Quantity	Expected	Server 1	Server 2	Server 3
HybridSBD A0 (D = 384, L = 4)	Latency (ms)	30.4	29.3 (-3.6 %)	29.1 (-4.3 %)	28.8 (-5.3 %)
HybridSBD A0 (D = 384, L = 4)	Peak VRAM (MB)	1,707	1,749.3 (+2.5 %)	1,702.4 (-0.3 %)	1,749.3 (+2.5 %)
HybridSBD A0 (D = 384, L = 4)	GFLOPs	29.4	29.4 (+0.1 %)	29.4 (+0.1 %)	29.4 (+0.1 %)
HybridSBD A4 (reduced)	Latency (ms)	10.7	10.6 (-1.4 %)	11.0 (+2.5 %)	10.2 (-4.4 %)
HybridSBD A4 (reduced)	Peak VRAM (MB)	941	976.6 (+3.8 %)	953.0 (+1.3 %)	976.6 (+3.8 %)
HybridSBD A4 (reduced)	GFLOPs	10.8	10.8 (-0.1 %)	10.8 (-0.1 %)	10.8 (-0.1 %)
AutoShot	Latency (ms)	151	241.8 (+60.1 %)	288.8 (+91.3 %)	160.1 (+6.0 %)
AutoShot	Peak VRAM (MB)	16,205	16,204.9 (-0.0 %)	16,182.9 (-0.1 %)	16,204.9 (-0.0 %)
AutoShot	GFLOPs	96.8	96.8 (+0.0 %)	96.8 (+0.0 %)	96.8 (+0.0 %)

HybridSBD reproduces on all three servers within 6 % (server 3: –5.3 % / –4.4 % latency, +2.5 % / +3.8 % VRAM). **AutoShot's latency reproduces only on server 3 (+6 %)**; on servers 1 and 2 it is 60 % and 91 % higher while its memory and operation count are exact, so it is the same model on a slower host.

5. Why AutoShot depends on the host

- **Most of its time is host-side Python.** Profiling one forward pass at B = 16 on server 2 (cProfile, 292 ms) shows two calls of AutoShot's own `gather_nd` (a port of `tf.gather_nd` in `supernet_flattransf_3_8_8_8_13_12_0_16_60.py`, called once in its `FrameSimilarity` layer and once in `ColorHistograms`) taking 285 ms, of which `Tensor.tolist()` is 93 ms; the 46 convolutions take 2 ms of host time in total. `gather_nd` copies about 0.6 million indices from the GPU to the CPU, turns them into nested Python lists and indexes with them, so it grows with the batch size and with the host's Python speed. On server 3 the same two calls take 157 ms.
- **Host measurements** (single thread):

	Server 2	**Server 3**
Pure-Python loop, 3 million iterations	0.62 s	0.11 s
Launch of a tiny CUDA op	6.7 μs	2.5 μs
AutoShot `gather_nd`, 2 calls at B = 16	285 ms	157 ms
AutoShot forward at B = 16 (benchmark)	288.8 ms	160.1 ms

- **Server 2's CPU does not run at its rated speed.** The Threadripper PRO 7975WX is rated 5.35 GHz, but `amd-pstate-epp` (governor `powersave`) has `scaling_max_freq` = 2.3 GHz and the busiest cores ran at 2.28 GHz. `/sys` is read-only inside the container and the files are not writable, so the cap is a host setting that cannot be changed from there; the CPU quota (about 30.7 CPUs) is not a limit for these runs. Server 1's EPYC reported a 1.9 GHz maximum. Server 3's i9-14900K was not capped.
- **Not the cause:** repeating the AutoShot run gave the same value (242.6 and 243.4 ms), the GPU was not throttled (SM clock 2,820–2,827 MHz, no throttle reasons), and `cudnn.benchmark`, TF32, cuDNN-disabled and channels-last settings did not move it toward 151 ms.

6. Notes

- **Script.** The task's `benchmark_model.py` is `benchmarks/speed.py` after the repository restructure; its `ours` and `baselines` commands run the protocol, `gpu-busy` produced the GPU-busy columns, `summary` builds the table. I added only the thin wrappers below, GFLOPs per clip in the result files and `gpu-busy`.
- **Weights.** TransNet V2, PERSIST and AutoShot use randomly initialised weights (latency, memory and operation counts do not depend on them; AutoShot's `ckpt_0_200_0.pth` and the PERSIST release checkpoints were not downloaded). A0 and A4 use their trained weights.
- **TransNet V2.** Official class with default arguments; input uint8 `[B, T, 27, 48, 3]`. Its `forward` always casts to float, permutes to `[B, 3, T, H, W]` and divides by 255; kept and timed. 7,616,258 parameters.
- **AutoShot.** `TransNetV2Supernet`; input float32 `[B, 3, T, 27, 48]` in 0–255 as in the paper's measurement; 14,299,202 parameters.
- **PERSIST.** Built from `configs/persist/persist_eval.py` through the official registry: 13,195,594 trainable parameters (13.20 M reported). Native input is uint8 `[B, 3, T, 112, 112]` (its test pipeline), native window T = 64; T = 128 also runs. The timed forward pass is the official inference path without post-processing: `ActionDataPreprocessor.preprocess` (cast and normalise), `Recognizer3D.extract_feat(test_mode=True)` (the backbone, which resizes 112 × 112 to 27 × 48 internally) and `SBDOursHead.forward`, returning the boundary probability. The cast and the internal resize are always run by the official code,

so they are kept; excluded are `predict()`'s copies to the CPU and debug fields, and video decoding. Built with `mmcv-lite` (pure Python, no compiled operators), which is enough to run it.

- **PERSIST at B = 1 on server 1** had a large spread (SD 17.5 ms; minimum 6.8, maximum 64.9 ms): a few passes were much slower, the mean is reported as measured. On server 3 the SD is 0.01 ms.
- **GFLOPs** are identical on all servers and equal the paper's (A0 29.43, A4 10.79, AutoShot 96.83).

7. Commands and wrapper code

```
# environment (server 3 and the paper's session: PyTorch 2.14.1+cu132; server 2 used the preinstalled 2.11.0+cu128)
pip install torch==2.14.1 torchvision --index-url https://download.pytorch.org/whl/cu132
pip install numpy tqdm scipy einops mmengine==0.10.7 mmcv-lite==2.1.0 opencv-python-headless matplotlib packaging pyyaml rich termcolor ya
pf addict importlib_metadata
git clone https://github.com/soCzech/TransNetV2 && git clone https://github.com/linty5/PERSIST && git clone https://github.com/wentaozhu/A
utoShot   # then check out the commits above
pip install --no-deps -e PERSIST/mmaction2

# measurement (code/ = hybridsbd/ + benchmarks/ of this repository; ckpt/<run>/best_model.pt = the study checkpoints)
export AUTOSHOT_DIR=AutoShot TRANSNETV2_DIR=TransNetV2/inference-pytorch PERSIST_DIR=PERSIST
PY='taskset -c 0-15 python'   # server 3 only: pin to the performance cores
$PY benchmarks/speed.py ours --device cuda --batch_sizes 1 16 32 --checkpoints ckpt/A0/best_model.pt ckpt/A4/best_model.pt --output out/ou
rs.json
$PY benchmarks/speed.py baselines --device cuda --batch_sizes 1 16 32 --output out/baselines.json
$PY benchmarks/speed.py gpu-busy --checkpoints ckpt/A0/best_model.pt ckpt/A4/best_model.pt --models autoshot transnetv2 persist --output o
ut/gpu_busy.json
$PY benchmarks/speed.py summary --dir out
```

Wrapper and loaders from `benchmarks/speed.py`:

```
def load_transnetv2(device: torch.device):
    """Official TransNetV2 (PyTorch, ``inference-pytorch``) with random weights: latency and memory do not depend on them."""
    sys.path.insert(0, str(TNV2_DIR))
    from transnetv2_pytorch import TransNetV2

    model = TransNetV2().to(device).eval()
    print("    TransNetV2 instantiated with random weights")
    return model
```

```
class PersistForward(torch.nn.Module):
    """The official PERSIST inference path without post-processing, as one forward pass.

    Input: uint8 ``[B, 3, T, H, W]``. The official data preprocessor (cast to float and normalise), the backbone (which
    resizes the frames to 27 x 48 itself) and the head are run exactly as ``Recognizer3D.predict`` runs them; the returned
    tensor is the head's boundary probability. Copying results to the CPU and the debug fields of ``predict`` are left out.
    """

    def __init__(self, recognizer: torch.nn.Module):
        super().__init__()
        self.recognizer = recognizer

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        batch, _ = self.recognizer.data_preprocessor.preprocess(list(x.unsqueeze(1)), None, False)  # [B, 1, 3, T, H, W]
        feats, kwargs = self.recognizer.extract_feat(batch, test_mode=True)
        return self.recognizer.cls_head.forward(feats, **kwargs)["probs"]


def load_persist(device: torch.device):
    """Official PERSIST model (``linty5/PERSIST``, MMAction2 fork) built from its evaluation config, random weights."""
    import warnings

    from mmaction.registry import MODELS
    from mmaction.utils import register_all_modules
    from mmengine.config import Config

    warnings.filterwarnings("ignore")
    register_all_modules()
    cfg = Config.fromfile(str(PERSIST_DIR / "configs" / "persist" / "persist_eval.py"))
    model = PersistForward(MODELS.build(cfg.model)).to(device).eval()
    print("    PERSIST built from configs/persist/persist_eval.py with random weights")
    return model
```

Raw results: `server_results/latency_baselines/` (server 1), `latency_baselines_server2/`, `latency_baselines_server3/` (`ours.json`, `baselines.json`, `gpu_busy.json`, `lscpu.txt`, `nvidia_smi.txt`, diagnostics).